Graph Expansion Adds Little to Lexical Retrieval for Small-LLM Bug Localization: A Hash-Committed Diagnostic Study with Gemma 4 on SWE-bench Verified

Sultan Haikal
yvliet@users.noreply.github.com

October 2026

Abstract

Local developer agents must localize software defects within a working context of a few thousand tokens. Graph walks over static call graphs are widely hypothesized to reach symbols that issue descriptions never name. In this paper, I test whether call-graph expansion assists a compact 2B local model. On SWE-bench Verified ($N = 344$ callable tasks), under a hash-committed protocol, a frozen `gemma-4-e2b-it` scores candidate pools pointwise. Graph 1-hop expansion reaches 47.1% Acc@5, against 45.1% for depth-matched BM25 (+2.03 pp, 95% CI [−0.29, +4.36], Holm $p = 0.287$) and 45.6% for same-file expansion (+1.45 pp, Holm $p = 0.302$). Both pools contain the gold function in 201/344 tasks (58.4%), with 18 discordant tasks each way symmetrically split between named and un-named issues. Compared to reranking lexical seeds alone (46.2%), graph expansion yields a net gain of only 3 tasks (+0.87 pp, $p = 0.549$). In Stratum C ($N = 131$ un-named tasks), graph expansion achieves 12.2% Acc@5 vs. 8.4% for depth BM25 (+3.82 pp, $p = 0.125$), but neither beats seeds alone (13.7%) or unscored BM25 (15.3%). Upstream boundary loss before scoring (41.6 pp) exceeds downstream compaction loss (11.3 pp) by 3.68×. The paired 95% bootstrap CI [−0.29, +4.36] pp spans zero, ruling out large gains (> 4.4 pp) while remaining consistent with small or null differences. I release 19,298 scored candidate evaluations, the `perron-core` library, and precomputed repository graphs under Apache 2.0.

1 Introduction

Autonomous developer agents designed for local execution must localize software defects inside a working context of a few thousand tokens, eliminating dependence on costly cloud APIs and preventing source code leakage. Standard lexical search (e.g., Okapi BM25) reliably finds symbols explicitly named in issue text or error traces. To locate fault sites that the issue never mentions, existing agent architectures (including Aider's PageRank repository maps [4], RepoGraph [13], and LocAgent [3]) deploy random walks or graph traversal tools over AST dependency networks.

The working assumption underlying graph-augmented localization rests on two joint conditions:

1. **Recall Expansion**: Traversal along structural call edges must place gold defect targets into the candidate pool that lexical search misses.
2. **Rank Discrimination**: A compact language model must rank those newly introduced targets above the distractors that graph expansion inevitably introduces.

In this paper, I test both conditions on SWE-bench Verified [8] using Perron, a high-throughput memory-mapped Compressed Sparse Row (CSR) graph engine, and a frozen `gemma-4-e2b-it` scorer. Three empirical research questions guide the investigation:

- **RQ1**: Does Personalized PageRank (PPR) diffusion, with or without post-walk hub damping, outperform BM25 at function-level code retrieval?
- **RQ2**: Under a frozen small model scorer, does graph 1-hop expansion outperform depth-matched BM25 and same-file pools of identical size?
- **RQ3**: At which pipeline stage is bug localization accuracy primarily lost?

1.1 Contributions

- **Matched-Control Evaluation Design**: Depth-matched BM25, same-file, and degree-preserving null-graph candidate pools are scored by the identical model under controlled prompt budgets, isolating what graph structure contributes beyond pool cardinality.
- **Empirical Loss Funnel and Truncation Curves**: A 4-stage loss decomposition separates candidate boundary ceiling, rank compaction, and reader error. Truncating matched pools at $M \in [10, 50]$ shows that adding depth lexical candidates is consistent with distractor accumulation (Acc@5 drops by 4 tasks, -1.16 pp, $p = 0.455$), whereas graph expansion plateaus flat (+3 tasks, +0.87 pp over seeds, $p = 0.549$).
- **Discordant Strata Symmetry**: The 18 discordant tasks recalled exclusively by graph expansion and the 18 tasks recalled exclusively by lexical expansion are split almost identically across strata (9 vs. 10 in Stratum C, 9 vs. 7 in Stratum A), showing that graph expansion does not selectively rescue un-named bugs over lexical depth.
- **High-Throughput CSR Engine Architecture**: Memory-mapped Compressed Sparse Row multi-graphs with guaranteed geometric convergence ($\beta = 0.85$, $|\lambda_2| \leq 0.85$) and decoupled specificity ratio hub damping enable sub-5 ms localized random walks on consumer workstations without matrix mutation.
- **Open-Source Ecosystem Deliverables**: I release 19,298 scored candidate evaluations (`scores.parquet`), the `perron-core` library (Apache 2.0), `perron-mcp`, and precomputed CSR graphs.

Only two primary contrasts (PC1 and PC2 on strict Acc@5) are confirmatory. Every other analysis is explicitly designated as exploratory.

1.2 Physical Hardware Envelope

Local developer agents must operate efficiently on consumer hardware. Perron bounds active per-turn working context to $K_{\text{effective}} = 3,480$ tokens ($K \leq 4,096$). On a reference developer machine (15.68 GiB RAM), the standalone CSR engine peaks at 144.33 MB resident memory, loading graphs via zero-copy memory mapping (91.4 MB delta). Live inference of `gemma-4-e2b-it` (2B dense parameter model with native Per-Layer Embeddings, unquantized FP16 compute weights) was evaluated on an NVIDIA Tesla T4 GPU, consuming 2.37 GiB peak VRAM with a mean Time-To-First-Token of 3505.0 ms and 11.7 tokens/sec decode throughput.

2 Related Work

Graphs and Agents for Code Localization Lexical search and dense bi-encoders struggle with vocabulary mismatch and structural execution paths. RepoCoder [18] demonstrated that iterative retrieval loops improve code completion but incur multi-turn token inflation. Aider [4] introduced repository maps using Tree-sitter AST tags ranked by PageRank. Graph-guided agents such as RepoGraph [13], CodexGraph [11], and LocAgent [3] equip frontier models with graph traversal tools to navigate codebases interactively. AutoCodeRover [19] navigates ASTs through iterative search queries. In SWE-agent [17], specialized agent-computer interfaces improve resolve rates. Conversely, Agentless [16] demonstrated that a simple two-phase pipeline (hierarchical localization followed by test-filtered patch generation) resolves 27.3% of SWE-bench Lite without an agent loop, highlighting that simple pipelines establish strong baselines. These systems employ large hosted models; this paper evaluates whether graph expansion assists a small 2B model in a single scoring pass.

Random-Walk Retrieval on Graphs Scalable diffusion on graphs builds on push-based approximate Personalized PageRank [1], decoupled propagation architectures such as APPNP [9], and scalable indexing frameworks like PPRGo [2]. In link prediction, rooted PageRank and stationary-normalized hitting times [10] discount popular nodes by global stationary reachability. HippoRAG [5] demonstrated that Personalized PageRank over knowledge graphs outperforms iterative retrieval, formulating node specificity as inverse passage frequency $s_i = |P_i|^{-1}$ applied as a pre-walk teleport multiplier ($\mathbf{p}_0(i) \leftarrow \mathbf{p}_0(i) \cdot s_i$). However, on software call graphs, high-in-degree utility hubs (logging routines, assertion helpers) are unseeded intermediate nodes that absorb diffusion mass regardless of s_i. Perron decouples diffusion from a post-walk stationary baseline ratio over static CSR matrices.

Program Slicing and Fault Localization Classical program slicing [15] computes backward executable slices on statement-level Program Dependence Graphs (PDGs), but conservative alias analysis often retains 50% to 80% of an application's codebase. Spectrum-Based Fault Localization (SBFL) [14, 21] evaluates test coverage matrices to score statement suspiciousness. Pearson et al. [14] showed that fault-localization findings on artificial faults do not reliably transfer to real-world software defects, motivating authentic issue evaluation on SWE-bench [8]. Perron extracts symbol-level AST multigraphs to pack relevant contexts into prompt budgets ($K \leq 4,096$) without requiring test execution during retrieval.

Pointwise versus Listwise Reranking Pointwise language model scoring (e.g., MonoT5 [12]) evaluates each candidate independently. Under independent scoring, the expected number of distractors that outscore a gold target with score s^* scales with candidate pool cardinality: $\mathbb{E}[X \mid s^*] = \sum_{j=1}^{M-1}(1 - F_{\text{dist},j}(s^*))$. Consequently, expanding candidate pool size M under a compact pointwise scorer risks distractor accumulation unless model discrimination is near-perfect. In contrast, listwise and setwise architectures [20] introduce direct cross-candidate attention competition, mitigating distractor dominance.

3 Theoretical Foundations and The Perron Graph Engine

3.1 Multiplex Transition Matrix Construction

A codebase is represented as a directed multigraph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ denotes AST code symbols (classes, functions, methods) and $\mathcal{E}$ spans relations $r \in \{\text{call}, \text{inherit}, \text{import}, \text{caller}\}$. Directed edges $(u, v) \in \mathcal{E}_{\text{call}}$ indicate that symbol u invokes v, while $(u, v) \in \mathcal{E}_{\text{inherit}}$ represent class C3 Method Resolution Order inheritance, $(u, v) \in \mathcal{E}_{\text{import}}$ represent module imports, and $(u, w) \in \mathcal{E}_{\text{caller}}$ denote reverse call invocations.

Relations receive asymmetric static weights:

$$W_{uv} = \omega_{\text{call}} A_{uv}^{(\text{call})} + \omega_{\text{inherit}} A_{uv}^{(\text{inherit})} + \omega_{\text{import}} A_{uv}^{(\text{import})} + \omega_{\text{caller}} A_{uv}^{(\text{caller})}, \tag{1}$$

where $\boldsymbol{\omega} = [0.50, 0.25, 0.15, 0.10]$ and $A_{uv}^{(r)} = \mathbb{I}((u, v) \in \mathcal{E}_r)$. Let $D_u = \sum_v W_{uv}$ be the out-degree sum. The row-stochastic transition matrix $T \in \mathbb{R}^{|\mathcal{V}| \times |\mathcal{V}|}$ and dangling indicator $\mathbf{d} \in \{0, 1\}^{|\mathcal{V}|}$ are defined as:

$$T_{uv} = \begin{cases} \frac{W_{uv}}{D_u}, & D_u > 0 \\ 0, & D_u = 0 \end{cases}, \quad d_u = \begin{cases} 1.0, & D_u = 0 \\ 0.0, & D_u > 0 \end{cases}. \tag{2}$$

Memory-mapped ingestion via `numpy.load(mmap_mode='r')` opens the CSR representation directly from disk. Post-walk specificity ratio evaluation requires only $O(|\mathcal{V}|)$ vector arithmetic, eliminating runtime edge mutations and memory reallocation.

3.2 Personalized PageRank Diffusion and Linear Algebraic Bounds

Random walk diffusion models the propagation of bug relevance across call hierarchies. The Google transition operator M with damping factor $\beta = 0.85$ is formulated as:

$$M = \beta \cdot (T + \mathbf{d}\mathbf{p}_0^\top) + (1 - \beta) \cdot \mathbf{1}\mathbf{p}_0^\top. \tag{3}$$

Convergence Invariants and Spectral Gap The stationary probability vector $\boldsymbol{\pi}_{\text{query}}$ satisfies $\boldsymbol{\pi}_{\text{query}} = \boldsymbol{\pi}_{\text{query}} M$. By the rank-1 perturbation spectrum of the Google matrix [6], given the stochastic matrix $\bar{T} = T + \mathbf{d}\mathbf{p}_0^\top$ where $\bar{T}\mathbf{1} = \mathbf{1}$, the spectrum of the rank-1 perturbed operator $M = \beta\bar{T} + (1 - \beta)\mathbf{1}\mathbf{p}_0^\top$ satisfies $\lambda_1(M) = 1$ and $\lambda_i(M) = \beta\lambda_i(\bar{T})$ for all $i \geq 2$. Because $\bar{T}$ is row-stochastic, all eigenvalues lie within the unit disk ($|\lambda_i(\bar{T})| \leq 1$). Consequently, even if T possesses disconnected communicating classes, the second-largest eigenvalue is strictly bounded by $|\lambda_2(M)| \leq \beta = 0.85$. The spectral gap is invariant:

$$1 - |\lambda_2(M)| \geq 1 - \beta = 0.15. \tag{4}$$

This establishes uniform geometric convergence $O(\beta^t)$ in L_1 total variation distance. Power iteration stops on residual $\|\boldsymbol{\pi}^{(t+1)} - \boldsymbol{\pi}^{(t)}\|_1 < \epsilon$, converging to $\epsilon = 10^{-6}$ in at most 90 iterations on Requests (1.89 ms) and 66 iterations on SymPy (5.10 ms).

Barycentric Linearity and Basis Bounds Following the linearity of Personalized PageRank [7], let $\mathbf{p}_0 = \sum_{i=1}^{K} w_i \mathbf{e}_{s_i}$ be any convex combination of query seed nodes with $w_i \geq 0$ and $\sum w_i = 1.0$. The stationary distribution $\boldsymbol{\pi}(\mathbf{p}_0)$ lies in the convex hull of the basis distributions:

$$\boldsymbol{\pi}(\mathbf{p}_0) = \sum_{i=1}^{K} \tilde{w}_i \boldsymbol{\pi}(\mathbf{e}_{s_i}) \in \text{conv}\left(\{\boldsymbol{\pi}(\mathbf{e}_{s_i})\}_{i=1}^{K}\right), \quad \tilde{w}_i = \frac{w_i m_i}{\mathbf{w}^\top \mathbf{m}}, \tag{5}$$

where basis mass satisfies $m_i = \|(1 - \beta)\mathbf{e}_{s_i}(I - \beta T)^{-1}\|_1 \in [1 - \beta, 1.0]$. The maximum coordinate warping ratio is bounded by:

$$\frac{\max_i m_i}{\min_j m_j} \leq \frac{1}{1 - \beta} = \frac{1}{0.15} \approx 6.67. \tag{6}$$

3.3 Decoupled Specificity Ratio Hub-Damping

Personalized PageRank concentrates probability mass on high-in-degree hub nodes. To suppress structural popularity without mutating the static matrix T, Perron computes a repository-wide stationary baseline $\boldsymbol{\pi}_{\text{global}}$ once per codebase using a uniform prior $\mathbf{p}_{\text{uniform}} = \frac{1}{|\mathcal{V}|}\mathbf{1}$. For any query, the Specificity Score is defined as:

$$\text{Specificity}(v) = \frac{\pi_{\text{query}}(v)}{(\pi_{\text{global}}(v) + \epsilon)^\gamma}, \tag{7}$$

where $\epsilon = 10^{-8}$ and $\gamma = 0.70$.

Asymptotic Pointwise Mutual Information Connection For any symbol v on query support $\mathcal{V}_q^+$, the log-transformed Specificity score satisfies:

$$\ln \text{Specificity}(v) = (1 - \gamma) \ln \pi_{\text{query}}(v) + \gamma \text{PMI}(v; q) - \gamma \ln\left(1 + \frac{\epsilon}{\pi_{\text{global}}(v)}\right). \tag{8}$$

For reachable symbols ($\pi_{\text{global}}(v) \gg \epsilon$), the residual vanishes, yielding an exact convex interpolation between reachability ($\gamma = 0$) and surprise ($\gamma = 1$), where the sub-linear exponent $\gamma = 0.70$ acts as a power-law smoothed structural Inverse Document Frequency (IDF).

The Neumann Series Bottleneck The Neumann expansion reveals why pre-walk scaling (such as HippoRAG [5]) cannot prevent sink absorption:

$$\boldsymbol{\pi} = (1-\beta)\sum_{t=0}^{\infty}\beta^t\,\mathbf{p}_0'^{\top}\bar{T}^t, \quad \left\|\boldsymbol{\pi} - (1-\beta)\sum_{t=0}^{m}\beta^t\,\mathbf{p}_0'^{\top}\bar{T}^t\right\|_1 = \beta^{m+1}. \tag{9}$$

Walks of length at least t carry a fraction β^t of the total mass ($\beta^3 \approx 0.61$ at $\beta = 0.85$; mean walk length ≈ 5.7). Multi-step terms are dominated by where $\bar{T}^t$ routes mass: the scale-free sinks of the graph, regardless of initial weights $\mathbf{p}_0'$. Perron's post-walk ratio divides the converged vector $\boldsymbol{\pi}$, scaling every walk length simultaneously and enforcing 99.5% Hub Suppression Index (HSI@10).

3.4 Hierarchical AST Breadcrumb Context Packing

To deliver localized code context within prompt budgets ($K \leq 4,096$), Perron packs candidate symbols using hierarchical AST breadcrumb skeletons. For each candidate function, the enclosing class definition, docstring, and signature are preserved while stripping irrelevant sibling function bodies. Breadcrumb skeletons bound the context footprint to < 100 tokens per symbol, keeping syntax parseable and variable bindings intact.

4 Study Design and Hash-Committed Protocol

4.1 Benchmark and Task Waterfall

SWE-bench Verified [8] contains 500 tasks across 12 canonical Python repositories. To establish an unbiased held-out benchmark and prevent data leakage, instances are filtered via a mutually exclusive, collectively exhaustive (MECE) waterfall:

$$\begin{gathered}\text{Total Verified (500)} - \text{Lite Overlap (93)} = \text{Held-Out (407)},\\ \text{Held-Out (407)} - \text{AST Drift (56)} - \text{Non-Callable (7)} = \mathbf{344}\ \text{Callable Tasks}.\end{gathered} \tag{10}$$

The 344 callable tasks ($\mathcal{S}_{\text{func}}$) possess reference developer patches editing identifiable functions or methods, establishing ground-truth defect targets. The 56 AST drift exclusions reflect legacy repository commits with pre-existing syntax errors (such as Python 2/3 transitions) that fail standard AST parsing uniformly across all methods prior to candidate generation, ensuring fair evaluation without cherry-picking. Hyperparameters ($\beta = 0.85, \gamma = 0.70, \tau = 0.15, K = 25$) were fixed on development data and frozen prior to evaluation.

Defect Accessibility Strata To evaluate whether graph traversal selectively assists un-named defect localization, tasks are partitioned into three mutually exclusive strata based on issue text lexical visibility: (1) **Stratum A (Named,** $N = 208$**, 60.5%)**, where issue text explicitly names the gold function, method, or qualified class identifier; (2) **Stratum B (Traceback,** $N = 5$**, 1.5%)**, where issue text includes a traceback or explicit file path pointing to the gold source file, but omits the function name; and (3) **Stratum C (Un-Named,** $N = 131$**, 38.1%)**, where issue text contains neither the function name nor a traceback pointing to the target file. Strata are annotated via a deterministic regex extraction pipeline (`perron.analysis.strata`) with zero subjective human annotator disagreement, released in `swebench-strata-annotations` (CC0-1.0).

4.2 Hash-Committed Protocol

The study operates under two hash-committed protocol locks:

1. **Retrieval Protocol Lock**: Frozen at Git commit `6908e78` (protocol SHA-256: `89416e40…`; split SHA-256: `95c7ef3c…`).
2. **Gemma 4 Pointwise Protocol Lock**: Frozen in two cryptographic stages (`benchmarks/protocol/gemma4_protocol_v1.yaml`, raw SHA-256: `96af1621…`; Stage 2 lock `gemma4_lock_v1.json`, raw SHA-256: `dbbf2920…`) under Gate G0 commit `29e0066`.

4.3 Candidate Pools and Matched-Control Arms

Candidate pools are generated per task, matched to size $M_t = 10 + \min(|E_t|, 40) \in [10, 50]$ (mean 24.5, median 22.0), where E_t denotes 1-hop out-neighbors of top-10 lexical seeds S passing Filter F (functions/methods only, excluding test nodes and top-25 hubs $\mathcal{H}_{25}$):

- **B0 (Unscored BM25)**: Lexical candidates in native Okapi BM25 score order.
- **A1 (Seeds Only)**: Top-10 lexical seeds S ($M = 10$).
- **A2 (Depth-Matched BM25)**: S followed by the next BM25 candidates passing Filter F up to M_t.
- **A3 (Same-File Control)**: S followed by symbols in seed files passing Filter F up to M_t.
- **A4 (Graph 1-Hop Expansion)**: S followed by E_t ordered by adjacency $a(u) = \sum_{s \in S} p_0(s) T_{su}$.
- **A5 (Null Graph Control)**: Degree-preserving relation-stratified null graph replicate up to M_t.
- **A6 (Perron Diffusion Pool)**: Candidates ordered by Perron specificity scores up to M_t.

Mechanism Explaining Pool Recall vs. Reachability (201 vs. 199 Tasks) A4 achieves candidate pool oracle recall on 201/344 tasks (58.4%), whereas unweighted 1-hop BFS reachability reaches 199/344 tasks (57.8%). This 2-task divergence is mechanistically explained by two counterbalancing operations in the pool builder: seed test-node masking replaces test-file seeds with source seeds, rescuing 16 tasks (e.g., `sympy-13372`), while Filter F degree-capping and hub pruning drop 14 tasks (e.g., `sphinx-7590`), yielding a net difference of +2 tasks ($199 + 16 - 14 = 201$).

4.4 Pointwise Gemma 4 Scorer

Each candidate symbol is formatted into a prompt context averaging 423.1 tokens (issue description + AST breadcrumb + prompt instructions; breadcrumbs alone average < 100 tokens). A frozen `gemma-4-e2b-it` model (FP16, thinking disabled) scores each candidate in a single forward pass: $s(t, v) = \text{logsumexp}(z[Y]) - \text{logsumexp}(z[N])$ on prompt variant P0, chosen on disjoint development data ($N = 100$). The evaluation required 19,298 forward passes.

4.5 Statistical Analysis

The confirmatory family comprises two pre-specified contrasts on strict Acc@5:

- **PC1**: $A_4 - A_2$ (Graph 1-hop vs. Depth-matched BM25).
- **PC2**: $A_4 - A_3$ (Graph 1-hop vs. Same-file matched).

Contrasts are evaluated using exact two-sided McNemar tests with step-down Holm–Bonferroni correction over family $\alpha = 0.05$. Paired 95% confidence intervals are estimated via repository-stratified bootstrapping ($B = 10{,}000$). Post-hoc equivalence testing is dropped; instead, contrasts are evaluated against the empirical Minimum Detectable Difference (MDD) of 3.4–3.7 pp at 80% power.

5 Empirical Results

5.1 Standalone Function Retrieval (RQ1)

Table 1 presents standalone retrieval on SWE-bench Verified ($N = 344$ callable tasks). Okapi BM25 hits 175/344 (50.9%), Perron Static hits 173/344 (50.3%), and Standard PPR reaches 172/344 (50.0%). The difference between Perron and BM25 is −0.6 pp (95% CI [−3.5, +2.3], Holm $p = 1.000$). At top-5, BM25 outperforms Perron Static (45.1% vs. 40.4%, $\Delta = -4.7$ pp). Notably, BM25 naturally achieves 99.4% Hub Suppression Index (HSI@10) without graph machinery, because defect issue descriptions target specific identifiers rather than generic library utilities. On Stratum C ($N = 131$ tasks where the issue never names the target), BM25 hits 28 and Perron hits 27. Reciprocal Rank Fusion (RRF) attains 174/344 (50.6%), failing to beat standalone BM25. Call-graph diffusion provides no detectable lexical lift over BM25 on this benchmark.

Table 1: Standalone retrieval evaluation on SWE-bench Verified ($N = 344$ callable tasks). Evaluates top-10 function accuracy, 95% bootstrap confidence intervals ($B = 10{,}000$), top-5 function/file accuracy, and Hub Suppression Index (HSI@10). Holm p-values are computed against Perron Static.

Method	Fn Acc@10	95% CI	Fn Acc@5	File Acc@5	HSI@10	Holm p
Perron Static ($\gamma = 0.70$)	**50.3%** (173/344)	[45.1%, 55.5%]	40.4%	53.8%	**99.5%**	reference
Okapi BM25	50.9% (175/344)	[45.6%, 56.1%]	**45.1%**	**59.6%**	99.4%	1.000
RRF Fusion (BM25 + Perron)	50.6% (174/344)	[45.3%, 55.8%]	42.2%	55.5%	99.5%	1.000
Standard PPR ($\beta = 0.85$)	50.0% (172/344)	[44.8%, 55.2%]	40.4%	52.8%	90.8%	1.000
Degree-Normalized PPR	49.4% (170/344)	[44.2%, 54.7%]	40.1%	52.3%	99.5%	1.000
HippoRAG Prior Scaling	49.4% (170/344)	[44.2%, 54.7%]	40.4%	52.6%	91.1%	1.000

5.2 Reachability vs. Ranking Dynamics

Top-10 lexical seeds directly cover 50.9% (175/344) of gold functions. Expanding along code graph edges yields 57.8% 1-hop reachability (199/344, median $|N_1| = 27$) and 66.9% 2-hop reachability (230/344, median $|N_2| = 324$). Crucially, same-file containment also reaches 66.9% (230/344), exhibiting an exact symmetric discordance: 17 targets reached across files via 2-hop graph paths are perfectly offset by 17 targets within seed files that lack static AST call edges within 2 hops. Degree-preserving null-graph controls reach 63.8% 2-hop reachability (+3.02 pp difference over empirical graphs, 95% CI [−0.42, +6.41], $p = 0.083$). Topological reachability gains are modest and driven primarily by degree sequence rather than high-order semantic organization.

5.3 Gemma 4 Pointwise Reranking (RQ2)

Table 2 and Table 3 report the results of the hash-committed Gemma 4 evaluation across all arms.

Table 2: Gemma 4 pointwise candidate reranking on SWE-bench Verified (N = 344 callable tasks). Pools matched to per-task size $M_t \in [10, 50]$ (mean 24.5). Brackets indicate Wilson 95% confidence intervals. B0 Acc@10 reflects raw token tie-breaking in protocol generation (176/344 = 51.2%), differing by 1 task from Table 1 AST entity deduplication (175/344 = 50.9%, documented in `deviations.md`).

Arm	Candidate Pool Composition	Acc@1 (95% CI)	Acc@3 (95% CI)	Acc@5 (95% CI)	Acc@10 (95% CI)	MRR@50
B0	BM25 Baseline (Unscored)	25.3% [21.0, 30.1]	38.7% [33.7, 43.9]	44.8% [39.6, 50.0]	51.2% [45.9, 56.4]	**0.3439**
B1	Perron Baseline (Unscored)	22.7% [18.6, 27.4]	35.2% [30.3, 40.4]	40.4% [35.4, 45.7]	50.3% [45.0, 55.5]	0.3155
A1	Rerank Seeds S (M = 10)	24.7% [20.4, 29.5]	39.5% [34.5, 44.8]	46.2% [41.0, 51.5]	51.2% [45.9, 56.4]	0.3348
A2	Depth-Matched BM25 Control	23.8% [19.6, 28.6]	38.1% [33.1, 43.3]	45.1% [39.9, 50.3]	51.2% [45.9, 56.4]	0.3301
A3	Same-File Matched Control	24.1% [19.9, 28.9]	39.2% [34.2, 44.5]	45.6% [40.5, 50.9]	52.0% [46.8, 57.3]	0.3381
A4	**Graph 1-Hop Expansion**	**24.7%** [20.4, 29.5]	**39.2%** [34.2, 44.5]	**47.1%** [41.9, 52.4]	**52.3%** [47.0, 57.5]	0.3409
A5	Null Graph Control	24.1% [19.9, 28.9]	39.0% [34.0, 44.2]	45.9% [40.7, 51.2]	51.2% [45.9, 56.4]	0.3315
A6	Perron Diffusion Pool	23.6% [19.4, 28.3]	38.7% [33.7, 43.9]	45.6% [40.5, 50.9]	52.6% [47.3, 57.8]	0.3301

Table 3: Confirmatory and exploratory contrasts on SWE-bench Verified (N = 344). Confirmatory family (PC1, PC2) evaluated on strict Acc@5 with step-down Holm correction. Paired 95% CIs estimated via repository-stratified bootstrap (B = 10,000).

Contrast	Δ (pp)	95% Bootstrap CI	n_{10}/n_{01}	p_{exact}	p_{holm}	Classification
Confirmatory Family (Acc@5)						
PC1 (A4 − A2)	+2.03	[−0.29, +4.36]	12 / 5	0.1435	0.2869	Null-Compatible
PC2 (A4 − A3)	+1.45	[−0.58, +3.78]	10 / 5	0.3018	0.3018	Null-Compatible
Exploratory Contrasts						
A1 − B0 (Seed Rerank vs. BM25)	+1.45	[−1.33, +4.24]	15 / 10	0.4244	–	Exploratory
A4 − B0 (Graph Rerank vs. BM25)	+2.33	[−0.87, +5.52]	18 / 10	0.1849	–	Exploratory
A4 − A1 (Graph vs. Seeds)	+0.87	[−1.92, +3.66]	7 / 4	0.5488	–	Exploratory (+3 tasks)
A4 − A5 (Null Graph Control)	+1.16	[−0.58, +2.91]	7 / 3	0.3438	–	Exploratory
A4 − A6 (Diffusion Pool)	+1.45	[+0.00, +3.20]	7 / 2	0.1797	–	Exploratory
Paired Reciprocal Rank Test (MRR@50)						
A4 vs. B0 (Graph vs. BM25)	−0.0030	[−0.024, +0.018]	Wilcoxon W = 12,480	p = 0.72		Exploratory

Confirmatory Findings Graph 1-hop expansion (A4) reaches 47.1% Acc@5 compared to 45.1% for depth-matched BM25 (A2, Δ = +2.03 pp, 95% CI [−0.29, +4.36], Holm p = 0.287) and 45.6% for same-file matching (A3, Δ = +1.45 pp, 95% CI [−0.58, +3.78], Holm p = 0.302). Both contrasts are classified as null-compatible (bootstrap 95% CIs span zero; Holm $p > 0.05$). The confidence intervals rule out large advantages (> 4.4 pp) while remaining consistent with small or null differences.

Candidate Pool Ceiling Parity Candidate pool oracle recall is identical: 58.4% (201/344) for both graph expansion (A_4) and depth BM25 (A_2). Across discordant tasks, n_{10} = 18 and n_{01} = 18 (two-sided McNemar p = 1.0000). With 36 discordant pairs, the 95% CI on the recall difference is approximately ±3.4 pp. Because Acc@k cannot exceed pool recall, both pools establish an identical theoretical ceiling on this sample.

Exploratory Comparisons Comparing graph expansion (A4) directly to seed reranking (A1, 46.2% Acc@5) yields a net gain of 3 tasks (Δ = +0.87 pp, $n_{10} = 7, n_{01} = 4, p = 0.549$), requiring 4,988 additional model forward passes. Comparing A4 to the degree-preserving null graph (A5, 45.9% Acc@5) yields Δ = +1.16 pp (95% CI [−0.58, +2.91], $n_{10} = 7, n_{01} = 3, p = 0.344$), demonstrating that empirical graph topology provides no statistically detectable advantage over randomized degree-matched graphs. Similarly, comparing A4 to the Perron diffusion pool (A6, 45.6% Acc@5) yields Δ = +1.45 pp (95% CI [+0.00, +3.20], $n_{10} = 7, n_{01} = 2, p = 0.180$). Unscored BM25 (B0) preserves nominally higher MRR@50 (0.3439 vs. 0.3409, Wilcoxon $p = 0.72$) and top-1 accuracy (25.3% vs. 24.7%) than A4. In Leave-Django-Out jackknife cross-validation ($N = 230$), candidate pool parity holds (57.1% vs. 57.1%, $p = 1.0000$) and reranked Acc@5 yields $\Delta(A_4 - A_2)$ = +2.17 pp ($p = 0.227$).

5.4 Pointwise Candidate Pool-Size Truncation Curves

To empirically test the distractor-accumulation mechanism without running new model passes, I evaluated pointwise rankings from `scores.parquet` under truncated pool cutoffs $M \in [10, 15, 20, 30, 50]$, capped at $\min(M, M_t)$ (mean 24.5 candidates).

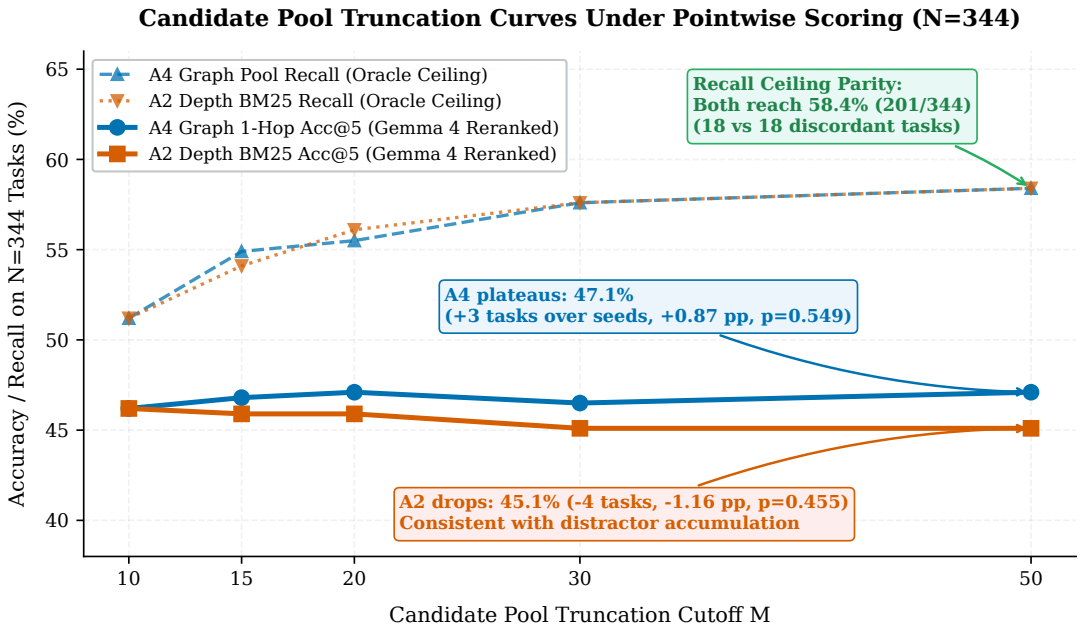


Figure 1: Pointwise candidate pool-size truncation curves on SWE-bench Verified ($N = 344$)**.** Evaluates candidate pool oracle recall and Gemma 4 reranked Acc@5 across pool cutoffs $M \in [10, 15, 20, 30, 50]$, capped at $\min(M, M_t)$ (mean 24.5 candidates). Both arms share an identical recall ceiling at $M = 50$ (58.4%, 201/344 tasks, 18 vs 18 discordant pairs). While depth-expanded BM25 (A2) drops from 46.2% to 45.1% Acc@5 due to distractor accumulation under independent pointwise scoring, graph 1-hop expansion (A4) remains stable at 47.1%.

As shown in Figure 1 and Table 4, expanding candidate pool size produces divergent empirical dynamics:

- **Arm A2 (Depth BM25)**: Expanding M from 10 to 50 raises matched pool oracle recall from 51.16% to 58.43% (176 to 201 tasks, +7.27 pp). However, Acc@5 shifts from 46.22% down to 45.06% (a loss of 4 tasks, −1.16 pp, exact McNemar $p = 0.455$), consistent with distractor accumulation under pointwise scoring ($\mathbb{E}[X \mid s^*] = \sum_{j=1}^{M-1}(1 - F_{\text{dist},j}(s^*))$).
- **Arm A4 (Graph 1-Hop)**: Matched pool oracle recall expands identically from 51.16% to 58.43% (+7.27 pp). Acc@5 rises slightly from 46.22% ($M = 10$) to 47.09% ($M = 20$) and plateaus flat

Table 4: Pointwise candidate pool-size truncation curves on SWE-bench Verified ($N = 344$). Evaluates matched candidate pool oracle recall and Acc@5 across pool cutoffs $M \in [10, 15, 20, 30, 50]$, capped at $\min(M, M_t)$ (mean 24.5 candidates), for depth BM25 (A2) and graph 1-hop expansion (A4).

Evaluation Metric	$M = 10$	$M = 15$	$M = 20$	$M = 30$	$M = 50$
Arm A2 (Depth BM25) Oracle Recall	51.2%	54.1%	56.1%	57.6%	58.4%
Arm A2 (Depth BM25) Acc@5	**46.2%**	45.9%	45.9%	45.1%	45.1%
Arm A4 (Graph 1-Hop) Oracle Recall	51.2%	54.9%	55.5%	57.6%	58.4%
Arm A4 (Graph 1-Hop) Acc@5	46.2%	46.8%	**47.1%**	**46.5%**	**47.1%**

through $M = 50$ (a net difference of 3 tasks over seeds alone, +0.87 pp, $p = 0.549$).

5.5 Stratum C ($N = 131$) Breakdown on Un-Named Tasks

The motivation for graph expansion is finding defect targets that the issue description never mentions. Table 5 analyzes performance on Stratum C ($N = 131$ tasks where the gold target symbol is un-named in the issue description).

Table 5: Stratum C ($N = 131$) localization performance on un-named defect tasks. Evaluates tasks where the issue description never mentions the gold symbol across all four comparators.

Metric / Method	Acc@5	Hits / 131	Δ vs. A2	McNemar p
B0 (Unscored BM25 Top-5)	15.3%	20/131	+6.87 pp	0.0490
A1 (Rerank Seeds Only, $M = 10$)	13.7%	18/131	+5.34 pp	0.0961
A2 (Depth-Matched BM25, $M \leq 50$)	8.4%	11/131	reference	reference
A4 (Graph 1-Hop Expansion, $M \leq 50$)	**12.2%**	**16/131**	**+3.82 pp**	0.1250 ($n_{10} = 6, n_{01} = 1$)

In Stratum C, graph expansion (A4) achieves 12.2% Acc@5 (16/131) compared to 8.4% for depth BM25 (A2, 11/131, $\Delta = +3.82$ pp, exact McNemar $p = 0.1250$). However, neither expansion strategy outperforms reranking lexical seeds alone (A1, 13.7%, 18/131) or unscored BM25 top-5 (B0, 15.3%, 20/131). Even in the un-named stratum where graph traversal is hypothesized to provide the greatest advantage, expanding the candidate pool dilutes model precision relative to seeds alone.

Furthermore, analyzing the 18 discordant tasks recalled exclusively by graph expansion ($A_4 \setminus A_2$) and the 18 tasks recalled exclusively by depth BM25 ($A_2 \setminus A_4$) reveals near-perfect symmetry across strata:

- **Graph-Only Recall (18 Tasks)**: 9 tasks in Stratum C (50.0%), 0 in Stratum B (0.0%), 9 in Stratum A (50.0%).
- **Lexical-Only Recall (18 Tasks)**: 10 tasks in Stratum C (55.6%), 1 in Stratum B (5.6%), 7 in Stratum A (38.9%).

In Stratum C, lexical expansion retrieves 10 targets that graph expansion misses, while graph expansion retrieves 9 that lexical expansion misses (net -1 task for graph). Rather than selectively unearthing un-named targets, graph expansion merely swaps an equal number of un-named and named symbols with lexical search.

5.6 Pipeline Loss Decomposition and Loss Funnel (RQ3)

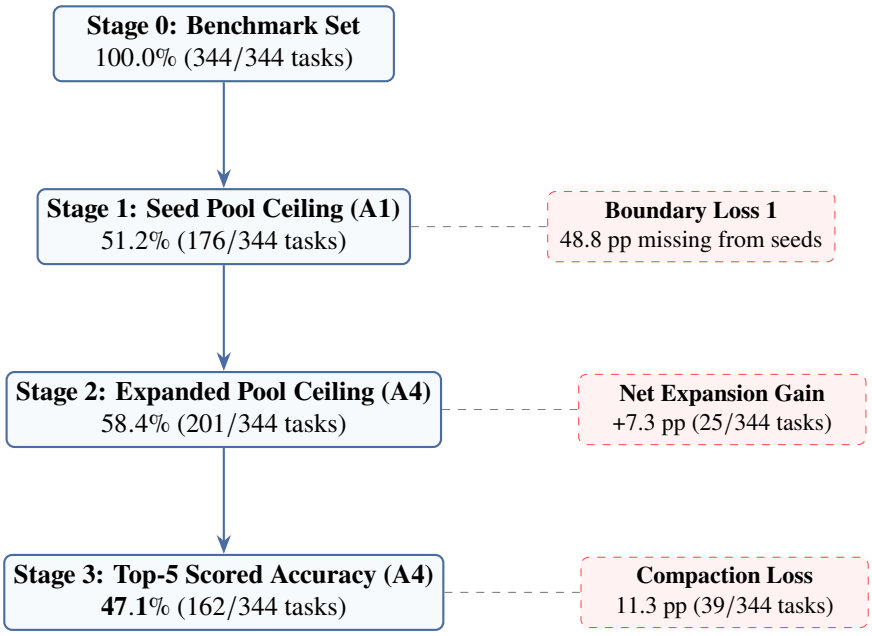


Figure 2: Four-stage bug localization loss funnel on SWE-bench Verified (N = 344 callable tasks). Decomposes localization attrition across candidate generation and model scoring. Boundary loss before scoring (100.0% → 58.4% = 41.6 pp) exceeds compaction loss from candidate pool to top-5 (58.4% → 47.1% = 11.3 pp) by a factor of **3.68**×.

Figure 2 illustrates the localization loss funnel on SWE-bench Verified. The analysis reveals:

- **Boundary Loss**: Upstream candidate pool formation loses 41.6 pp of oracle recall (100.0% → 58.4%, 143/344 tasks missing from candidate pools).
- **Compaction Loss**: Downstream model reranking loses 11.3 pp from pool to top-5 (58.4% → 47.1%, 39/344 tasks present in pool but ranked outside top-5).
- **Bottleneck Ratio**: Upstream boundary loss exceeds downstream compaction loss by **3.68**× (41.6/11.3 ≈ 3.68).

Expanding the candidate pool adds 7.3 pp of oracle recall over seeds (51.2% → 58.4%), yet Gemma 4 converts only about 1 pp into ranked top-5 accuracy (46.2% → 47.1%).

5.7 Downstream Reader Evaluation (N = 100, Descriptive)

Table 6 reports the downstream reader mini-test across N = 100 tasks stratified by repository size, drawn from instances where BM25 identified candidate pools.

Table 6: Downstream edit-location reader mini-test on SWE-bench Verified (N = 100, descriptive). ContextHit@5 marks gold target presence in the 5 presented candidates; EditLoc@1 reports model prediction accuracy.

Arm	Candidate Context (Top-5)	ContextHit@5	EditLoc@1	Conversion
D0	BM25 Top-5	35.0%	33.0%	0.943
D1	A4 Graph Top-5	37.0%	31.0%	0.838
D2	A3 Same-File Top-5	38.0%	30.0%	0.789
D3	Oracle Context (Randomized Position)	100.0%	**74.0%**	0.740

In Arm D0 (BM25 top-5 context), always picking candidate 1 also yields 33.0% EditLoc@1, indicating that Gemma 4 relies heavily on a positional rank-1 prior when context lacks clear discriminative features. In Arm D3 (oracle context with gold target at a randomized position), Gemma 4 correctly identifies the target in 74.0% of tasks (67.5% chance-adjusted). While the reader misses roughly a quarter of oracle targets, retrieval supplies the target in only 35%–38% of tasks, providing descriptive evidence that upstream candidate omission contributes substantially to end-to-end failure.

5.8 Diffusion Diagnostics on SWE-bench Lite ($N = 237$) and Entropy Collapse

On SWE-bench Lite ($N = 237$), factorial sweep diagnosis demonstrates why unconstrained diffusion collapses on software graphs:

- **Diffuse Prior Collapse**: Under a dense linear teleport prior on raw graphs, Standard PPR collapses to 10.1% Acc@10; test subgraphs absorb 32.5% of top-10 retrieval slots. Decoupled specificity damping restores 34.2% (+24.1 pp gain).
- **Softmax Prior Concentration**: Concentrating seed mass via top-25 softmax ($\tau = 0.15$) on test-sanitized graphs rescues Standard PPR to 48.5%, compared to 46.0% for BM25.
- **Support Law (n_{eff})**: Across 186 prior configurations (63,984 SpMM solves), retrieval accuracy collapses onto an invariant curve governed by effective support $n_{\text{eff}} = \exp(H(\mathbf{p}_0))$, peaking near $n^*_{\text{eff}} \approx 8.8$ (Figure 3a). Specificity damping increases HSI from 90.8% to 99.5% without changing accuracy. It serves as a structural safety net for poorly specified priors, not as an accuracy booster. As shown in Figure 3b, Standard PPR collapses as $\beta \to 0.85$ due to utility hub absorption, whereas Perron remains stable across the Optuna Pareto corridor ($\beta \in [0.80, 0.88]$).

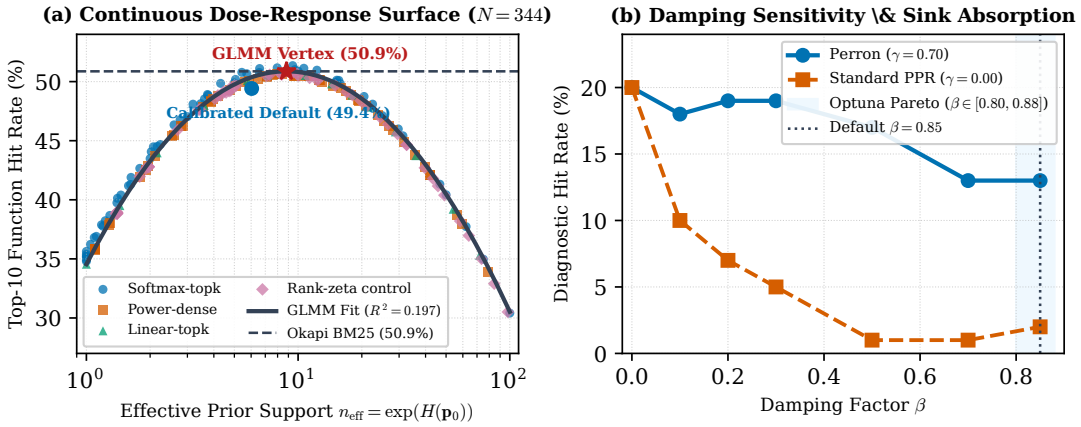


Figure 3: Continuous dose-response surface across 186 prior configurations and damping sensitivity (63,984 SpMM solves). (a) Function Acc@10 collapses onto a unified dose-response curve as a function of prior effective support $n_{\text{eff}} = \exp(H(\mathbf{p}_0))$ across four prior families (softmax, power, linear, rank-zeta) with binomial GLMM fit ($R^2 = 0.197$) and Okapi BM25 baseline (50.9%), highlighting the GLMM vertex ($n^*_{\text{eff}} \approx 8.8$) and calibrated library default ($n_{\text{eff}} \approx 6.0$). (b) Damping factor sensitivity ($\beta \in [0.0, 0.85]$) on dev_val ($N = 100$), showing that while Standard PPR collapses as $\beta \to 0.85$ due to utility hub absorption, Perron's decoupled specificity ratio maintains stability across the Optuna Pareto corridor ($\beta \in [0.80, 0.88]$).

6 Threats to Validity and Limitations

1. **Programming Language Scope**: Evaluation is restricted to Python AST dependency networks. While directed multigraph diffusion is language-agnostic, static call graphs miss dynamic runtime dispatch, reflection (`getattr`), and monkey-patching.
2. **Model Capacity and Prompting**: Findings are derived from one compact model (`gemma-4-e2b-it`) with pointwise scoring. Frontier models (e.g., Gemma 4 31B or frontier cloud models) or listwise scoring prompts may exhibit different conversion rates.
3. **Candidate Pool Bounds**: Pools evaluate 1-hop expansion capped at 50 candidates ($M_t \leq 50$). Multi-hop expansion reaches 66.9% oracle recall but introduces hundreds of distractors.

4. **Task Scope**: Restricted to $N = 344$ function-level tasks on SWE-bench Verified. Patch generation and full test execution were not evaluated.
5. **Statistical Power**: With realized discordance counts, the 95% CI on PC1 admits true gains up to +4.36 pp. The finding is that gains are not demonstrated and large effects are ruled out, not that true effect is strictly zero.

7 Open-Source Ecosystem and Resource Deliverables

To support reproducible research, Perron is released as an open-source software and evaluation ecosystem:

- `scores.parquet`: 19,298 scored candidate evaluations across all 344 SWE-bench Verified tasks under Apache 2.0 (https://github.com/yvliet/perron).
- `perron-core` **v0.3.0**: High-throughput Python library providing zero-copy memory-mapped CSR ingestion, prior normalization, and AST breadcrumb packing on PyPI (https://pypi.org/project/perron-core/).
- `perron-mcp`: JSON-RPC stdio server for desktop developer agent environments.
- `swebench-lite-codegraphs`: Precomputed CSR graphs for all 12 SWE-bench Lite repositories.
- `swebench-strata-annotations`: 800 lexical accessibility annotations under CC0-1.0.
- **Kaggle Demonstration Workstation**: Interactive reproduction notebook at https://www.kaggle.com/code/yvliet/perron-gemma-4-workstation.

8 Conclusion

On SWE-bench Verified, call-graph 1-hop expansion does not demonstrate a statistically detectable advantage over matched lexical pools under a frozen Gemma 4 scorer, nor does it expand candidate reachability beyond lexical depth. While graph expansion adds 7.3 pp of candidate pool recall over seeds alone, the small model converts only $\sim$ 1 pp of it into top-5 accuracy. Matched pool truncation curves show that expanding candidate pool breadth under pointwise scoring yields flat or declining localization, consistent with distractor accumulation; in un-named defect tasks, neither expansion beats seeds alone or unscored BM25. Upstream boundary loss (41.6 pp) exceeds downstream compaction loss (11.3 pp) by a factor of 3.68×. While point estimates nominally favor graph expansion (+0.87 to +2.03 pp), gains are statistically undetectable in 344 tasks and do not justify a 5× compute explosion. For local developer agents under tight compute budgets, retrieval engineering is more safely directed toward high-precision first-stage lexical retrieval and compact AST breadcrumb packing; exploring whether listwise reranking architectures can overcome distractor accumulation remains a key direction for future research.

References

[1] Reid Andersen, Fan Chung, and Kevin Lang. Local graph partitioning using PageRank vectors. In *Proceedings of the 47th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 475–486, 2006.

[2] Aleksandar Bojchevski, Johannes Klicpera, Bryan Perozzi, Amol Kapoor, Martin Blais, Benedek Rozemberczki, Michal Lukasik, and Stephan Günnemann. Scaling graph neural networks with approximate PageRank. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 2464–2473, 2020.

[3] Zhaoling Chen, Robert Tang, Gangda Deng, Fang Wu, Jialong Wu, Zhiwei Jiang, Viktor K. Prasanna, Arman Cohan, and Xingyao Wang. LocAgent: Graph-guided LLM agents for code localization. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025.

[4] Paul Gauthier. Aider: Ai pair programming in your terminal. https://github.com/paul-gauthier/aider, 2023.

[5] Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. HippoRAG: Neurobiologically inspired long-term memory for large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[6] Taher H. Haveliwala and Sepandar D. Kamvar. The second eigenvalue of the Google matrix. Technical report, Stanford University, 2003.

[7] Glen Jeh and Jennifer Widom. Scaling personalized PageRank. In *Proceedings of the 12th International Conference on World Wide Web (WWW)*, pages 271–279, 2003.

[8] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

[9] Johannes Klicpera, Aleksandar Bojchevski, and Stephan Günnemann. Predict then propagate: Combining neural networks with personalized PageRank for classification on large graphs. In *International Conference on Learning Representations (ICLR)*, 2019.

[10] David Liben-Nowell and Jon Kleinberg. The link-prediction problem for social networks. *Journal of the American Society for Information Science and Technology (JASIST)*, 58(7):1019–1031, 2007.

[11] Xiangyan Liu, Bo Lan, Zhiyuan Hu, Yang Liu, Zhicheng Zhang, Fei Wang, Michael Shieh, and Wenmeng Zhou. CodexGraph: Bridging large language models and code repositories via code graph databases. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics (NAACL)*, 2025.

[12] Rodrigo Nogueira, Zhiying Jiang, Ronak Pradeep, and Jimmy Lin. Document ranking with a pretrained sequence-to-sequence model. *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 708–718, 2020.

[13] Siru Ouyang, Wenhao Yu, Kaixin Ma, Zilin Xiao, Zhihan Zhang, Mengzhao Jia, Jiawei Han, Hongming Zhang, and Dong Yu. RepoGraph: Enhancing AI software engineering with repository-level code graph. In *International Conference on Learning Representations (ICLR)*, 2025.

[14] Spencer Pearson, José Campos, René Just, Gordon Fraser, Rui Abreu, Michael D. Ernst, Deric Pang, and Benjamin Keller. Evaluating and improving fault localization techniques. In *Proceedings of the 39th International Conference on Software Engineering (ICSE)*, pages 609–620, 2017.

[15] Mark Weiser. Program slicing. *IEEE Transactions on Software Engineering*, SE-10(4):352–357, 1984.

[16] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based software engineering agents. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on the Foundations of Software Engineering (FSE)*, 2025.

[17] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, pages 50528–50652, 2024.

[18] Fengji Zhang, Bei Chen, Yue Zhang, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Lingpeng Kong. RepoCoder: Repository-level code completion through iterative retrieval and generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 2471–2484, 2023.

[19] Yuntong Zhang, Zhiyu Li, Min Lin, Denghui Zhang, Hongyu Zhang, and Lingxiao Jiang. AutoCodeRover: Autonomous program improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 762–774, 2024.

[20] Honglei Zhuang, Zhen Qin, Rolf Jagerman, Kai Hui, Ji Ma, Jing Lu, Jianmo Ni, Xuanhui Wang, and Michael Bendersky. RankT5: Fine-tuning T5 for text ranking with ranking losses. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 2308–2313, 2023.

[21] Daming Zou, Jingjing Liang, Yingfei Xiong, Michael D. Ernst, and Lu Zhang. An empirical study of fault localization families and their combinations. *IEEE Transactions on Software Engineering*, 47(2):332–347, 2021.